

REPLY TO THE CREATION-API DOCUMENT OF 17 SEPTEMBER

Item 1 is done.

So are 2 to 6, and 8 to 14.

You asked for one thing above all others: that `/demo/*` stop letting a `jobId` spend our budget and publish to our channel. **That is built, tested and on a branch.** So is almost everything else on your list. This document says what changed, answers the six questions that were only ever questions, and names the four things that still need a person rather than a commit.

You were right on all three counts, and one of them was worse than you thought.

The `404` you measured was not a broken auth check — there was no auth on `/demo` at all, and no catch-all, so unmatched paths fell through to Express. The TTL was worse than "two hours": it was a timer armed at *creation*, so a produce run started 110 minutes into a job lost its record at 120 minutes **while it carried on spending**. And the rate limiting was inverted exactly as you described.

1 · What changed

Every row is covered by a test that fails if it regresses. The surface had none before this; it now has 48.

YOUR ITEM	WHAT IT DOES NOW
2.1 Authenticate <code>/demo/*</code>	Split by cost, not by route. Writing and reading a script buys nothing, so those stay open and your instructors can read before you spend. <code>/produce</code> and <code>/approve</code> require a bearer token. Separately, every job is bound to its creator — so a leaked <code>jobId</code> alone now reads nothing, produces nothing and publishes nothing.
2.2 Throttle <code>produce/approve</code>	Limits are per tenant , not per IP. Your point landed: a per-IP limit is a per-organisation limit for anyone behind one egress, so our "6 per hour" was six per hour for your whole university. Every <code>429</code> now carries <code>Retry-After</code> and <code>X-RateLimit-*</code> .
2.3 Idempotent <code>/produce</code>	<code>Idempotency-Key</code> accepted, scoped to (tenant, job, key), written <i>before</i> dispatch. A replay returns the original response with <code>Idempotency-Replayed: true</code> . Without a key, a repeat on a job already producing returns 202 with the existing run — the old <code>409</code> is gone.
2.4 Persist job state	Jobs are files, not a Map. They survive a restart today. They survive a <i>redploy</i> the moment a volume is attached — see §4, item A.
2.5 TTL from the terminal state	Seven days from the last transition . A job stuck in <code>producing</code> is reported, never deleted quietly.
2.6 Webhook or cursor	Both. <code>GET /demo/jobs?since=&cursor=</code> , and an optional <code>callbackUrl</code> at creation, HMAC-signed with the timestamp inside the signed payload.
2.8 Structured <code>by</code>	The tenant is recorded from your <i>credential</i> , not from the field. Your instructor's name is carried alongside it. Nothing you approve is filed under Aroma's name again.
2.9 / 2.10 The bridge	A job now names the catalogue key it will appear under; a catalogue row now carries <code>youtube.url</code> . See §3.
2.11 Health route	<code>GET /api/v1/health</code> , unauthenticated, no lesson data. Point a monitor at it and it will not go red when a token rotates.

2 · The six questions

These cost nothing to answer and should not have waited.

2.1 · Does the playable copy include the 2.6-second intro? (your items 9, \$2.10)

`atSeconds` already includes it. Do not add it yourself. It is measured from the start of `<videoId>_final.mp4` — the file we serve and the file we upload. `lessonAtSeconds` excludes it, and exists only so the two can never be silently confused.

Measured, not asserted. There is now a test that probes the real bumper and the real deliverable on every run:

<code>intro.mp4</code>	2.600 s	(ffmpeg, measured)
<code>bumper-durations.json</code>	2.600 s	(the constant the container uses)
<code>atSeconds - lessonAtSeconds</code>	2.600 s	(every checkpoint, every video)

Read `timing.introOffsetSource`. Never assume 2.6.

`probed` — measured from that video's own intro. `brand-constant` — the committed measurement, and the usual answer in production, where `.mp4` files are not in the image. `no-bumpers` — delivered bare, so the offset is genuinely 0 and `atSeconds` `=== lessonAtSeconds`; adding 2.6 to one of these fires every checkpoint 2.6 s late. `assumed` — and that is exactly the case where `timing.trusted` is `false`.

2.2 · Are slugs immutable? (your item 13)

Yes. What churned was existence, not identity. A slug is minted once, before the video is made, and is never rewritten. We went and checked `...-r2fi` specifically.

Here is what actually happened. Videos made through `/demo/make-video` are written into the container's filesystem at runtime, and our image build strips `out/`, `art/` and every `.mp4`. Exactly one such folder is committed to our repository. So a redeploy took them, and the slug was never touched. Because a re-run of the same topic draws a fresh timestamp suffix, a lesson re-made afterwards reappears under a new `videoId` — which from where you are standing is indistinguishable from a rename.

You should not have had to work that out from the outside, and you should not have to again. Every catalogue row now says so itself:

<code>"durability": "committed"</code>	in our repository; survives a redeploy. Safe to link.
<code>"durability": "ephemeral"</code>	made at runtime; WILL disappear. Do not link a block to it.
<code>"durability": "unknown"</code>	we could not tell. Treat as ephemeral.

Of the 24 rows in the catalogue today, 23 are `committed`. We would rather tell you `unknown` than guess: a row wrongly called durable is a dead question inside one of your courses, and a learner finds it before either of us does.

2.3 · What do `questionStyle` and `pausesVideo` mean? (your item 11)

QUESTIONSTYLE	PAUSESVIDEO	WHAT THE PLAYER MUST DO
popup	true	The question is never drawn and never spoken . The popup is the only place the learner will ever see it — skip it and they miss it entirely. Pause, ask, require an answer, show feedback, resume. This is every current video.
on-screen	false	Older format: the question and its answer are drawn into the video itself. Do not pause over these — the video answers itself. Use <code>onScreenUntilSeconds</code> if you want to mirror it.

`questionStyle` is derived, not stored: a video is `popup` when *every* one of its checkpoints has `rendersInVideo: false`. Your 17/8 split moving from 18/6 is the older on-screen videos being re-cut into the popup format.

2.4 · Is `timingTrusted: false` transient, and can it flip after linking? (item 12)

It can flip in both directions, and yes, after you have linked. Two independent causes, and they behave differently:

- A missing beat length. `durations.json` is written by the voiceover step; until it exists, or if any non-checkpoint beat is missing from it, every later time is a guess. This is *transient* — it becomes true when the video is next rendered.
- The intro offset was guessed (`introOffsetSource: "assumed"`), meaning we could neither probe the bumper nor read the committed constant. Rarer, and a misconfiguration on our side rather than a stage not yet run.

Your three untrusted videos changing identity between Tuesday and Wednesday is the first cause: those are videos mid-pipeline, not videos that broke. **Treat it as a live field, not a property of the link.** Re-read it when you serve, and refuse to fire a checkpoint whose `timing.trusted` is false at that moment, rather than at the moment you mapped it.

2.5 · `catalogueUpdatedAt`, and is the ETag payload-derived? (item 14)

The ETag is **payload-derived** — a SHA-1 of the exact serialised body — so it is stable while the content is and changes only when something real does. Two cautions, because we would rather you hear them from us than discover them at 100 rows:

- It is compared as a raw string. A **weak** (`W/"..."`) or multi-value `If-None-Match` will never match, and you will get a 200 every time.
- It covers the whole response including `count`, so any row changing changes the catalogue ETag.

`catalogueUpdatedAt` is now on every row, derived from the files the payload is actually built from, so it cannot claim a freshness the content does not have.

2.6 · The bridge between the two APIs (items 9 and 10)

You described this exactly right: two islands, and a human crossing by eye. Both halves are closed, in both directions.

```
# A job now names the catalogue key it will appear under –
"catalogue": {
  "videoId": "how-to-mark-a-register-ab12",
  "path": "made/how-to-mark-a-register-ab12",
  "checkpointsUrl": "/api/v1/videos/how-to-mark-a-register-ab12/checkpoints"
}

# – and a catalogue row now carries the playable copy.
"youtube": { "url": "https://youtu.be/...", "videoId": "...", "privacyStatus": "unlisted" },
"youtubeVideoId": "..."
```

Two fields are called `videoId` and they are not the same thing.

The row's `videoId` is our folder name and is what `/checkpoints` takes. `youtube.videoId` is a YouTube id. We nested it rather than flattening it for precisely this reason — join on `path`, never on `videoId`.

One honest limit: we have only ever recorded two publishes, both from August, and neither is in a folder that has checkpoints — so `youtube` will read `null` on every row today. Everything published from now on carries it. We are not going to backfill something we do not have the records for and pretend otherwise.

3 · What you will need to change

Small, and all of it in the direction you were already building.

1. Send your token on `/produce` and `/approve` . The same bearer you already use for `/api/v1/*` , or a second one if you would rather — say which and we will mint it.
2. Call `/claim` once between creating a job and producing it, if the job was created from a browser rather than your server. Server-to-server, you will not need it: a job created with your token is already yours.
3. Read `durability` before you bind a lesson block. This is the one that stops \$1 of your document happening again.
4. Drop the intro-offset arithmetic if you have any. `atSeconds` is already right.
5. Optionally, delete your poller and give us a `callbackUrl` . We need to allowlist your host first — send it and it is one variable.

Your \$3 items 3 and 6 can go.

Copying the MP4 out of our container, and the two-failed-lookups vanished-job detector, were both compensating for the TTL. Keep the job mirror, the script snapshot, the proxy and your own spend controls — we would keep all four in your position. The proxy in particular is still worth it even now that the `jobId` is no longer a credential.

4 · What is still open, and who has it

#	WHAT	WHO
A	Attach a Railway volume. Everything above works without it, but a redeploy still clears jobs until it exists. The code needs no change — the store moves onto the volume by itself.	AROMA
B	Rotate the API token and send the replacement outside chat. Your item 15, and you are right that it is overdue.	AROMA
C	A Taleemabad University YouTube channel (your \$2.7). We cannot solve this in code — it needs a channel and a credential we do not hold. Your offer to create one and hand us the credential is the cheapest answer and we would like to take it.	JOINT
D	Per-tenant budgets are built — your monthly ceiling is enforced before a run starts, and <code>GET /demo/spend</code> shows you what remains, so a 402 is never a surprise. What we still need from you is the number.	JOINT
E	<code>series</code> → module mapping, six series (your item 16). Still ours to send.	JOINT
F	The August content-write schema (your item 17). Open since 10 August; we have not forgotten it.	AROMA

On your \$2.2 footnote: thank you for not spoofing `X-Forwarded-For` . You were right that it would have worked — we trusted the whole header. That is fixed, and the limits that matter are keyed on your credential now rather than on an address, so the question does not arise again.

One thing to expect

The public demo page can no longer spend money on an anonymous click — it asks for a token at the Produce button. That is deliberate, and it is the same change as item 1. If you have handed that link to anyone, tell them it is now a way to read a script for free, not a way to make a video.